Seidel, Peter-M., "Multiple Path IEEE Floating-Point Fused Multiply-Add", Proc. 46th Int. IEEE MWSCAS, 2003

Multiple Path IEEE Floating-Point Fused Multiply-Add

PETER-MICHAEL SEIDEL
Computer Science and Engineering Department
Southern Methodist University
Dallas, TX, 75275
seidel@engr.smu.edu

Abstract - *We propose optimizations for the IEEE floating-point fused multiply-add operation by considering multiple exclusive parallel computation paths in the implementation. For the proposed design we can show a significant performance improvement over conventional implementations. Considering a variable latency implementation allows for further reduction of the average latency.*

I. INTRODUCTION & SUMMARY

Many contemporary floating-point units are based on the functionality of an IEEE floating-point multiply-add (FMA) operation [5,13,22]. The FMA functionality is not only used to implement floating-point addition, multiplication and fused multiply-add operations with IEEE rounding [6,7,15]. It is also a very useful basic functionality for the implementation of floating-point division and transcendental functions [2,19]. Few different implementations have been suggested for the IEEE FMA operation [5,7,8,13,22] all of them being very costly, because they combine hardware for floating-point addition and floating-point multiplication and require huge alignment shifters for wide intermediate representations. Because the FMA operation is used very frequently (for various purposes) with all of its hardware, its latency has a significant effect on the processors performance and its switching activity significantly contributes to the power dissipation of the processor. It is therefore important to optimize the performance and the power dissipation of the FMA implementation.

We are proposing micro-architectures for the FMA implementation to reduce latency over conventional implementations. Our optimizations are based on optimization techniques that have been proposed for floating-point addition and floating-point multiplication implementations and that are now applied and tuned for the FMA implementation.

The main optimization proposed is to consider multiple parallel computation paths for the implementation. This idea has been first introduced for floating-point addition in Farmwald's dual path algorithm that considers two exclusive parallel computation paths [4,18]. We extend this principle for the FMA operation by distinguishing between five cases with distinct parallel computation paths. In each of the five computation paths, part of the computation for the generic FMA operation can be saved, thus, saving delay on the critical path. On each of the five paths less hardware is active than in the general case. Because the decision between the five paths can be made early in the computation, this allows to deactivate the gates on the paths that are not used which could even allow to reduce the overall switching activity of the computation.

The latency varies among the five computation paths and allows to complete the FMA computation earlier in most of the cases. Similar properties have been used for variable latency implementation of floating-point addition and division [10,12,15]. The implementation of proposed FMA architecture with variable latency allows to further reduce the average latency of the FMA operation.

IEEE Rounding implementations have been optimized for floating-point addition and multiplication in [3,18,20] by the use of rounding injections. The use of rounding injections is adapted to the five computation paths for improving the IEEE rounding implementation of the proposed FMA implementation.

The main options in multiplication implementations at the micro-architectural level are the choice of the recoding scheme [1,14,17] and the topology of the adder tree. Recoding allows reducing the number of partial products to be compressed in the adder tree of the multiplier. Higher-radix recoding adds the delay for generating multiples of the multiplicand in advance. During the multiple computation of the multiplicand additional manipulations of the multiplier (2^{nd} operand) are possible (in addition to recoding) without increasing the effective latency of the operation. We are using such organization to hide part of the delay for pre-processing one of the multiplicative FMA operands.

Additionally, in recoded multipliers the recoded operand can be accepted in redundant (partially compressed) form [21]. We use redundant representations and partial compressions [9,11] for the addition of the rounding injections and the detection of leading zeros.

Our implementation is discussed for IEEE double precision operands implementing all four IEEE rounding modes. We estimate the critical path of our design to show that we improve the performance of the FMA operation compared to conventional implementations.

In Section 2 we introduce notation and review conventional implementations for IEEE FP FMA operation. In Section 3 we introduce multiple path selection criteria to prepare the discussion and evaluation of the implementation in Sections 4 and 5.

II. NAÏVE IMPLEMENTATION

We are considering the operation of IEEE floating-point fused multiply-add, that gets as an input three floating-point operands with values *opa*, *opb* and *opc* each being represented with a sign, a normalized significand in the range [1,2) and an exponent, so that

$$opa = (-1)^{sa} \cdot fa \cdot 2^{ea},$$
$$opb = (-1)^{sb} \cdot fb \cdot 2^{eb},$$
$$opc = (-1)^{sc} \cdot fc \cdot 2^{ec}.$$

The exponents are represented with n bits and the significands with p bits, where $(n,p) = (8,24)$ for single precision and $(n,p) = (11,53)$ for double precision.

The requested result is the normalized floating-point representation of the following expression after IEEE rounding in one of the four IEEE rounding modes:

$$result = rnd\left((-1)^{sa \oplus sb} \cdot fafb \cdot 2^{ea+eb} + (-1)^{sc} \cdot fc \cdot 2^{ec}\right)$$

The operation can also be considered as the floating-point addition of the two operands given as factorings $(sa \oplus sb, ea + eb, fa \cdot fb)$ and (sc, ec, fc). The sign of the effective operation is computed by $s.eff = sa \oplus sb \oplus sc$.

A naïve implementation of IEEE floating-point fused multiply-add operation considers the sequence of the following steps (we focus on the significand path):

1. Exponent difference: $\delta = ea + eb - ec$
2. Significand product: $fprod = fa \cdot fb$
3. Alignment shift: $fca = fc \cdot 2^{-\delta}$
4. Operand negation: $fcan = (-1)^{s.eff} fca$
5. Significand addition: $facc = fprod + fcan$
6. Complementation: $faccabs = |facc|$
7. Normalization: $faccn = norm\_sft(faccabs)$
8. Rounding: $frnd = round_{mode}(faccn)$
9. Post-Normalization: $fres = post\_norm(frnd)$

A simple implementation considers a sequential implementation of the sequence of computation steps from above. Such organization would be rather slow. In [5] and [8] optimizations over this simple organization have been considered. These optimizations include the parallel computation of exponent difference, alignment shift and significant multiplication and the parallel computation of binary compression and leading zero prediction of the sum representation. The optimizations in [8] include a reorganization of the normalization shift with part of the sum compression to allow for an earlier computation of rounding. Figure 1 illustrates a basic architecture of an implementation that is similar to the description from [5].

III. MULTIPLE PATH SEPARATION

As mentioned in the previous section, the floating-point FMA operation can be considered as the addition of a FP product with a FP operand. This interpretation suggests to consider the organization of optimized floating-point addition units for improved FP FMA implementations.

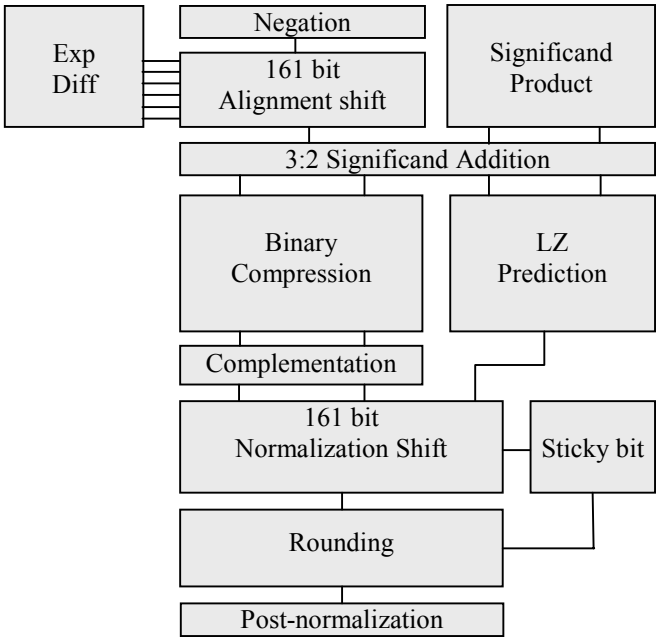

Fig.1. Structure of conventional FP FMA implementation

A survey of optimization techniques in high-performance FP addition is given in [18]. One major optimization technique considered in high-performance FP adders is the use of two parallel computation paths that are used exclusively for different ranges of the exponent difference with later selection. In each of the two computation paths the computations can be simplified allowing to reduce the complexity of or even eliminate various computation steps.

We apply a similar strategy for the computation of FP FMA, but use different path selection conditions and distinction between more than two different cases.

We consider five different cases that depend on the relative alignment of the significand product $fprod = fa \cdot fb$ with significand fc. Each case is illustrated and the corresponding conditions are listed in Figure 2 below:

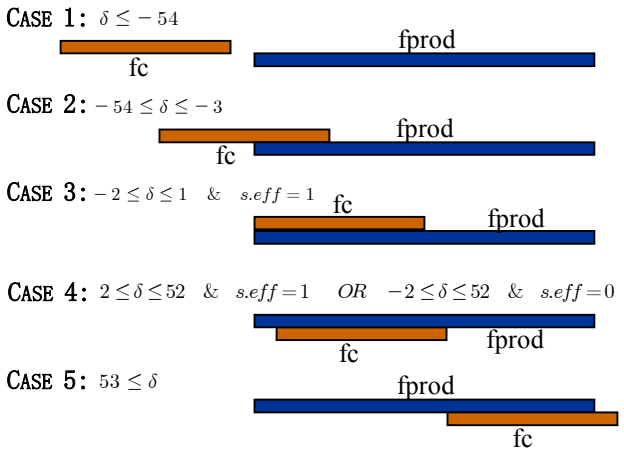

Fig. 2. Multiple Path Selection Criteria

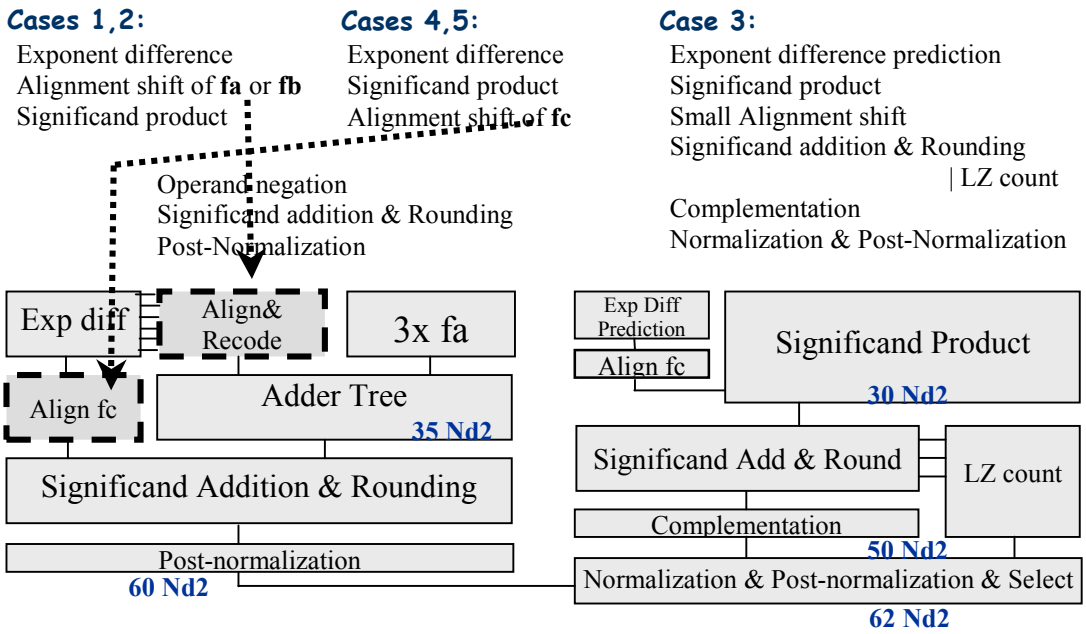

Fig. 3. Structure of proposed multiple path IEEE FP FMA implementation annotated with rough timing estimates for double precision operands.

IV. IMPLEMENTATION

For the implementation we separately consider the conditions in the five distinct cases:

In case 1 we consider the condition that the exponent difference is bound by $\delta \leq -54$. In this case the significand product *fprod* has only a very minor contribution to the result of the FMA operation and the computations reduce to steps 8 and 9 from Section 2:

$$fres = \begin{cases} post\_norm(fc + 2^{-53}) & \text{if mode=RI} \\ fc & \text{otherwise.} \end{cases}$$

In case 2 we consider the condition that the exponent difference is in the range $-54 < \delta \leq -3$. In this case the significand *fc* still determines the leading digits of the result. By aligning the product *fprod*, this allows to eliminate the need for steps 6 and 7 of the complementation and the normalization shift. Without steps 6 and 7 the significand addition from step 5 can easily be integrated with IEEE rounding from step 8. The fast parallel IEEE rounding implementation from [3] is feasible for this situation. Also the computation of steps 1 - 4 can be done in parallel.

In case 3 only effective subtractions are considered for the exponent difference in the small range $-2 \leq \delta \leq 1$. Like in the Near-path of the dual-path floating-point addition algorithm the small range allows the exponent difference to be predicted based on the two least significant bits of the exponents. The need for a large alignment shift is also eliminated in this way for this case. Because case 3 assumes effective subtractions, the negation of the operands can be made unconditional. The organization of the remaining computation steps from 5 to 9 requires some special attention. The need for a large normalization can only be caused by the cancellation of leading bits through the effective subtraction operation in this case. The maximum normalization shift distance is therefore 53 bit positions in this case. We have reorganized steps 5 - 9 to the sequence 5,8,6,7 and 9 considering integrated parallel computation of significand addition, rounding and leading zero count followed by integrated computation of normalization and post-normalization shift. We employ a special hardware implementation for the integrated addition and rounding implementation considering variable position rounding in a parallel prefix organization. A detailed discussion of this implementation is beyond the scope of this paper.

In case 4 the conditions for the exponent difference range ensure that the leading digits of the result are dominated by the representation of the significand product *fprod*. In this case alignment by less than 53 positions can be easily computed on *fc* during the significand multiplication. The exponent difference range also avoids the accumulation from becoming negative or that a large normalization shift becomes necessary and allows eliminating steps 6 and 7 from the computation.

In case 5 we consider the condition that the exponent difference is bound by $53 \leq \delta$. In this setting the significand *fc* only contributes to the rounding decision. The computation can therefore be organized in a slightly modified form of floating-point multiplication with IEEE rounding as in [3].

Considering the optimized organizations for each case, common substructures can be shared as illustrated in Figure 3 resulting in two major computation paths: one that is shared between cases 1,2,4 and 5 and the other one for an optimized implementation of case 3.

V. Evaluation & Comparison

We evaluate the latency of the proposed implementation using a delay model that is proposed in [8]. Delays are measured in units of NAND gate delays with a fanout of 2 (Nd2). We take from [8] the delay of the significand multiplication for double precision to be 30 Nd2 units using a Booth radix-4 multiplier. Because the adder tree is not fully populated for double precision with 53 bit significands, we integrate the addition of the aligned significand in case 3 with the adder tree compression. The implementation details for the integrated significand add and round computation could not be discussed with the given space limitation, but we estimate the delay of this step to be 20 Nd2 delays including the conditional complementation of the result. The normalization and post-normalization shift and the selection between the paths require 6 more stages of MUXES which we estimate to have a delay of 12 Nd2 units. The delay for the computation in cases 1,2,4 and 5 consists of the latency of a double precision Booth radix-8 multiplier with an estimated delay of 35 Nd2 units. For the remaining computations in these paths we assume the multiplier rounding implementation from [3], having an additional delay of 24 Nd2 units. With these estimations the paths are balanced and have a critical delay of 62 Nd2 units. In Table 1 we compare this latency with the latencies that are mentioned in [8] for the implementations proposed in [5] and [8] for double precision. It seems that these delay estimations do not consider a post-normalization shift in the implementation that would result in an additional delay of 6 Nd2 units.

Separate optimization of the computations in cases 5 and 1 allow further reduction of the implementation latency for these cases as listed in Table 1. A variable latency implementation of FP FMA could make use of these faster paths and make their results available earlier to other processing units, thereby further reduce the average latency of the implementation.

TABLE 1:
COMPARISON OF LATENCIES FOR DIFFERENT FP FMA IMPLEMENTATIONS

Implementation	Latency	Improvement
IBM paper [5]	**87(+6) Nd2**	0%
Bruguera&Lang [8]	**72(+6) Nd2**	-17% (-16%)
Proposed Unified implementation	**~62 Nd2**	-29% (-33%)
Case 5	**~52 Nd2**	-40% (-44%)
Case 1	**~18 Nd2**	-79% (-81%)

VI. References

[1] A. D. Booth. *A signed binary multiplication technique.* Quart. Journ. Mech. and Applied Math. Vol. 4. No. 2. pp. 236-240. (1951).

[2] G. Even, P.-M. Seidel and W.E. Ferguson. *A Parametric Error Analysis of Goldschmidt's Division Algorithm*, Proc. IEEE Int. Symp. on Computer Arithmetic (ARITH16), 2003.

[3] G. Even and P.M. Seidel. A comparison of Three Rounding Algorithms for IEEE Floating-Point Multiplication. IEEE Trans. Computers. Vol. 49. No. 7. pp. 638-650. (2000).

[4] M.P. Farmwald. *On the design of high performance digital arithmetic units.* PhD thesis, Stanford University, 1981.

[5] E. Hokenek, R. Montoye and P.W. Cook. Second-Generation RISC Floating Point with Multiply-Add Fused. IEEE J. Solid-State Circuits. Vol. 25, No. 5, pp. 1207-1213. (1990).

[6] *IEEE standard for binary floating-point arithmetic.* ANSI/IEEE754-1985, New York, (1985).

[7] R.M. Jessani and M. Putrino. Comparison of Single- and Dual-Pass Multiply-Add Fused Floating-Point Units. IEEE Trans. Computers. Vol. 47, No. 9. pp.927-937. (1998).

[8] T. Lang and J.D. Bruguera. Floating-Point Fused Multiply-Add with Reduced Latency. IEEE Conference on Computer Design (ICCD), (2002).

[9] D.W. Matula and A.M. Nielsen. *Pipelined Packed-Forwarding Floating Point: I. Foundations and a Rounder.* In Proceedings of the 13th Symposium on Computer Arithmetic, Vol. 13, pp. 140-147, (1997).

[10] S.M. Müller. *On the Scheduling of Variable Latency Fuctional Units.* In Proc. of the 11th ACM Symposium on Parallel Algorithms and Architctures (SPAA'99), pp.148-154, (1999).

[11] A.M. Nielsen, D.W. Matula, C.N. Lyu, and G.Even. *Pipelined Packed-Forwarding Floating-Point: II. An Adder.* In Proceedings 13th Symposiumon Computer Arithmetic, pp. 148-155, (1997).

[12] S.F. Oberman and M.J. Flynn. *A Variable Latency Pipelined Floating-Point Adder.* In Proceedings EUROPAR'96 Parallel Processing, volume LNCS 1124, pp. 183-192, (1996).

[13] F.P. O'Connell and S.W. White. *POWER3: The Next Generation of PowerPC Processors.* IBM J. Research and Development. Vol. 44, No. 6. pp. 873-884. (2000).

[14] W.J. Paul and P.-M. Seidel. *To Booth or Not To Booth?,* INTEGRATION, the VLSI journal, Jan. 2003.

[15] P.-M. Seidel. *Floating-Point Interlock Collapsing by Value Prediction, Partial Forwarding and Fused Operations*, manuscript submitted to IEEE ICCD 2003.

[16] P.-M. Seidel. *Operand Modification Schemes for Reduced Power Multiplication, Proceedings of the 36th Asilomar Conference on Signals, Systems & Computers, pp. 52-56, 2002.*

[17] P.-M. Seidel, L. McFearin D.W. Matula. *Binary Multiplication Radix-32 and Radix-256,* Proceedings of the 15th IEEE International Symposium on Computer Arithmetic (Arith15), IEEE Computer Society Press, pages 23-32, 2001.

[18] P.-M. Seidel and G. Even, *On the Design of Fast IEEE Floating-Point Adders, Proceedings of the 15th International Symposium on Computer Arithmetic (Arith15),* IEEE Computer Society Press, pages 184-194, 2001.

[19] P.-M. Seidel, *Exact Arithmetic based on Floating-Point Numbers, Proceedings Int. Symposium on Scientific Computing, Computer Arithmetic and Validated Numerics (SCAN2000),* Karlsruhe, Germany, Sept.2000.

[20] P.-M. Seidel, *On the Architecture of IEEE Compliant Floating-point Units*, Proceedings of the 18th IASTED Conference on Applied Informatics, pp. 260-268, (2000).

[21] P.-M. Seidel. *High-Speed Redundant Reciprocal Approximation*, INTEGRATION, the VLSI, Journal 28(1999), pp. 1-12.

[22] H. Sharangpani and K. Arora. *Itanium Processor Microarchitecture.* IEEE Micro Mag. Vol. 20, No. 5. pp. 24-43. (2000).